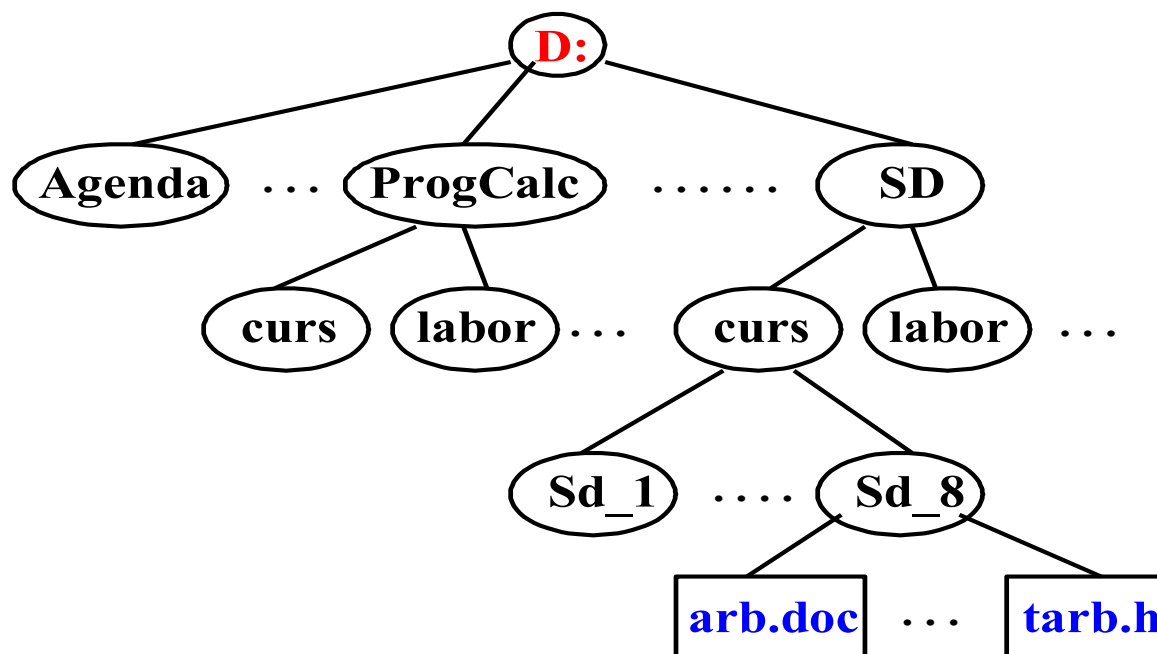


Structuri ierarhice: Arbori

O entitate oarecare poate fi **indivizibilă** sau **structurată**, în marea majoritate a cazurilor, ca o **ierarhie** de componente.

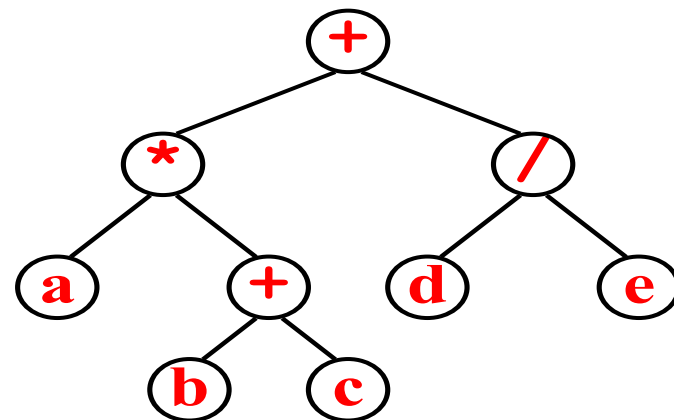
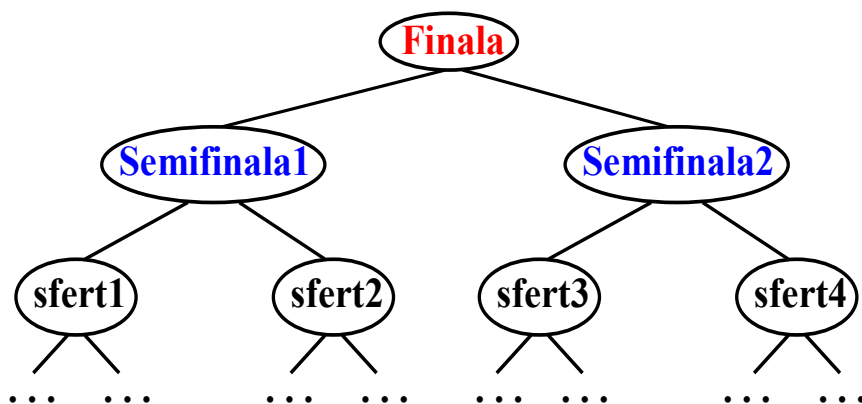
Exemple:

- informația înregistrată pe suport magnetic



Structuri ierarhice: Arbori

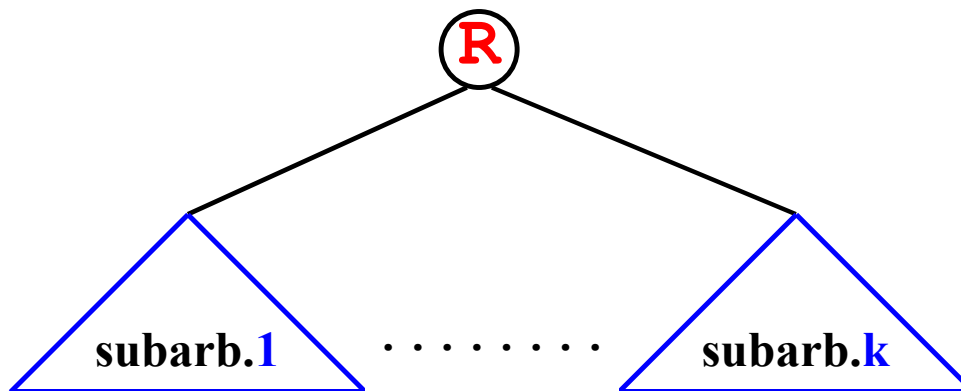
- cuprinsul unei cărți
- organizarea administrativă a societăților comerciale, ministerelor, universităților etc.
- programul meciurilor dintr-un turneu eliminatoriu
- arborele unei expresii aritmetice



Arbori

Un **arbore** este format dintr-un nod **rădăcină**, căruia îi este atașat un **număr finit de arbori**. Pentru a evidenția relația ierarhică, aceștia sunt numiți **subarbori**.

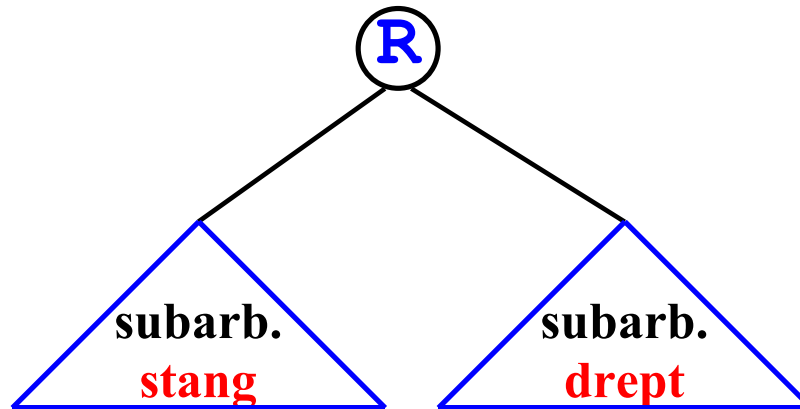
Orice nod dintr-un arbore poate fi privit ca rădăcină a unui (sub)arbore.



Arbori

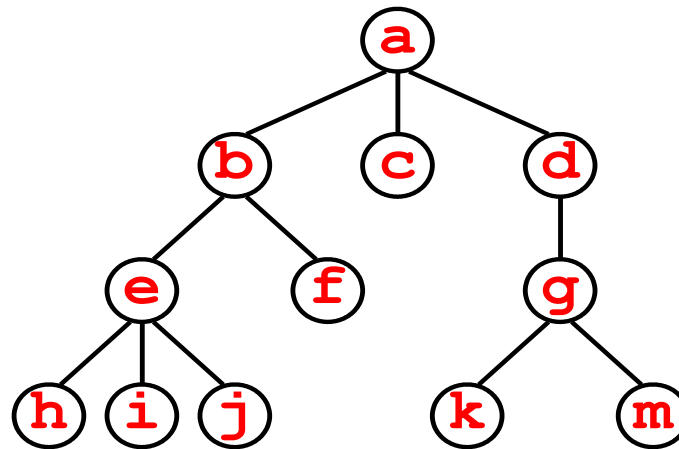
Termeni uzuali:

- **arbore vid** - nu conține nici un nod
- **arbore binar** - nodurile au **cel mult 2** subarbori
 - subarbore **stâng**
 - subarbore **drept**



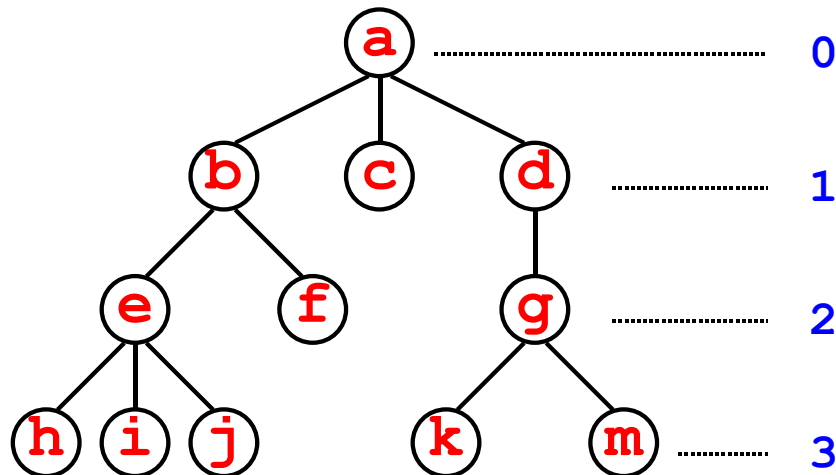
Arbori

- **arbore multicăi** – arbore în care nodurile pot avea mai mult de 2 subarbori



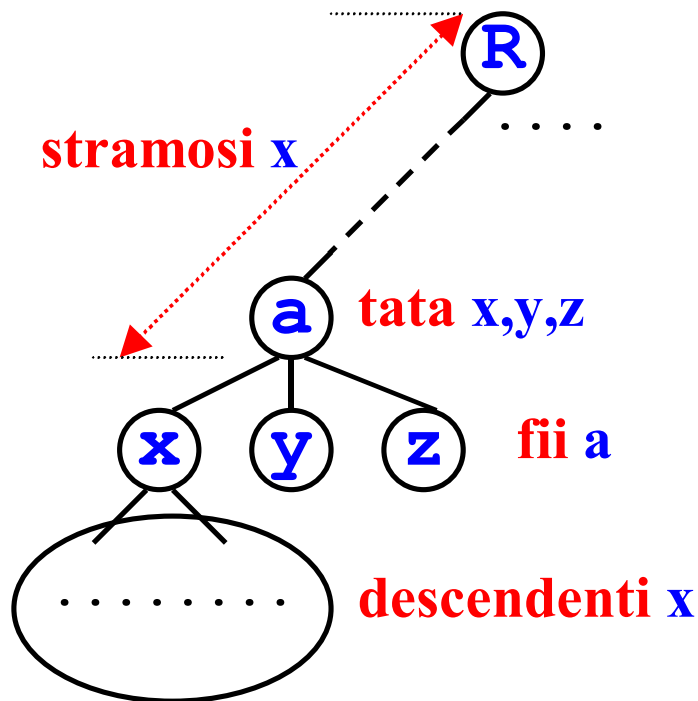
Arbori

- **ordinul** (gradul) unui nod - numărul de subarbori atașați nodului
- **noduri interne** - nodurile care au subarbori
- **noduri externe** (noduri terminale sau **frunze**) – nodurile fără subarbori (cu ordin 0)
- **nivelul** unui nod – distanța la care se află față de radacină (radacina are nivel 0)
- **înălțimea** unui arbore – nivelul maxim din arbore

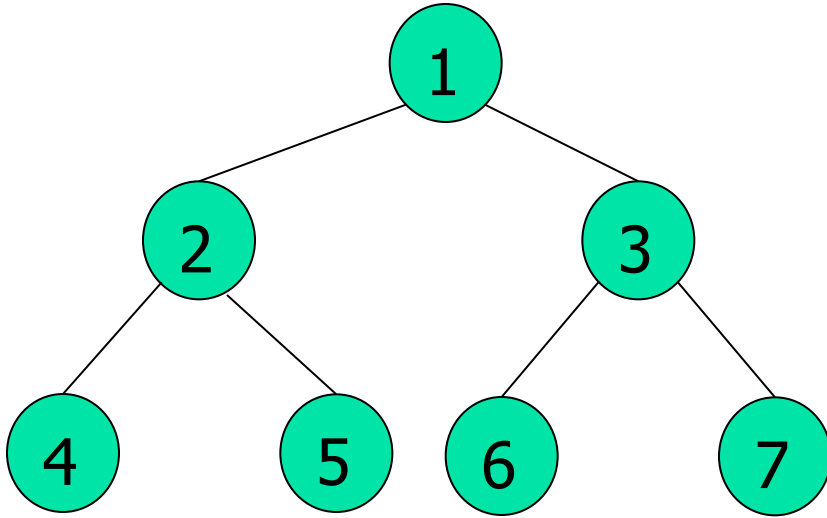


Arbori

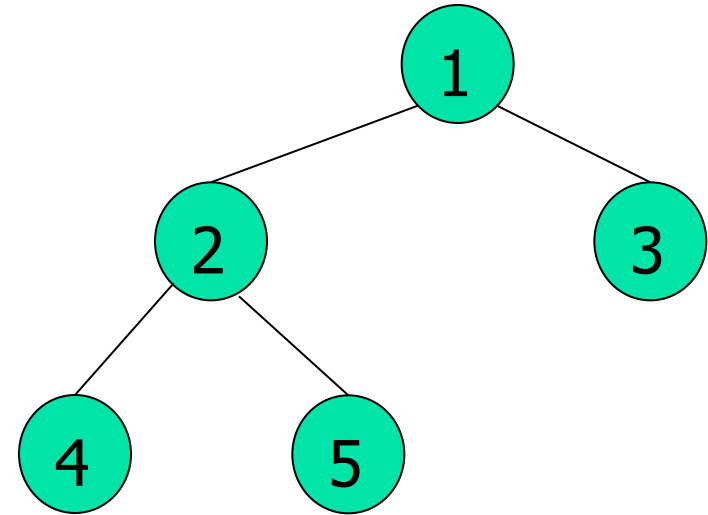
- **tată / fiu** – noduri aflate la distanța 1 (numite și **strămoș / descendent direct**)
- **frați** (siblings) – fiii aceluiași nod



Arbori

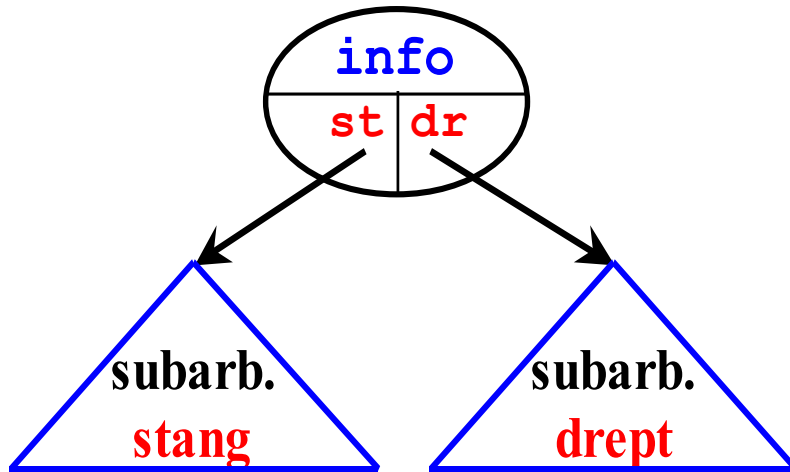


arbore plin



arbore complet

Tipul arbore binar



```
typedef struct nod
{
    int info;
    struct nod *st, *dr;
} TNod, *TArb, **AArb;
```

Adresare :

informatia: `a->info`

subarborii: `a->st`

`a->dr`

Teste arbore binar

Frunză

```
((a)->st == NULL && (a)->dr == NULL)
```

Nod interior

```
((a)->st != NULL) || ((a)->dr != NULL)
```

Teste arbore binar

Ordin nod

$((a) \rightarrow \text{st} \neq \text{NULL}) + ((a) \rightarrow \text{dr} \neq \text{NULL})$

frunza: $0 + 0 \Rightarrow 0$

numai subarbore stang: $1 + 0 \Rightarrow 1$

numai subarbore drept: $0 + 1 \Rightarrow 1$

doi subarbori: $1 + 1 \Rightarrow 2$

Numar noduri

```
int NrNoduri(TArb r)
```

```
/* numarul de noduri din arborele r -*/
```

```
{
```

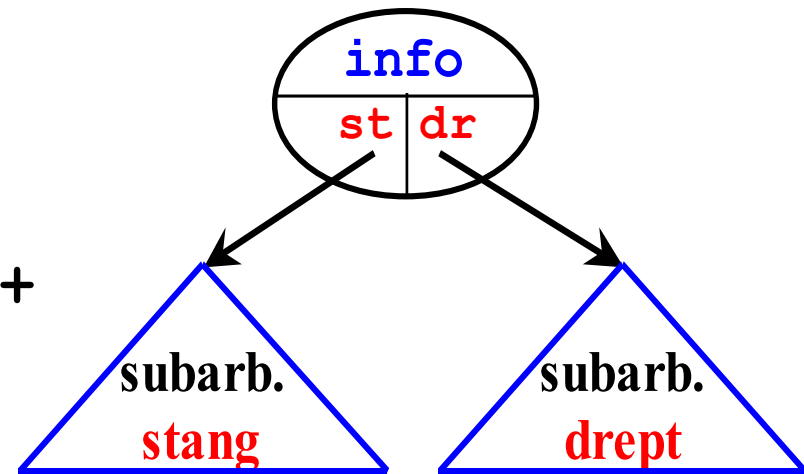
```
    if (!r) return 0;
```

```
    return 1 +
```

```
        NrNoduri(r->st) +
```

```
        NrNoduri(r->dr) ;
```

```
}
```



Numar niveluri

```
int NrNiveluri(TArb r)
```

```
/* numar niveluri (0 pentru arbore vid) */
```

```
{
```

```
    int ns, nd;
```

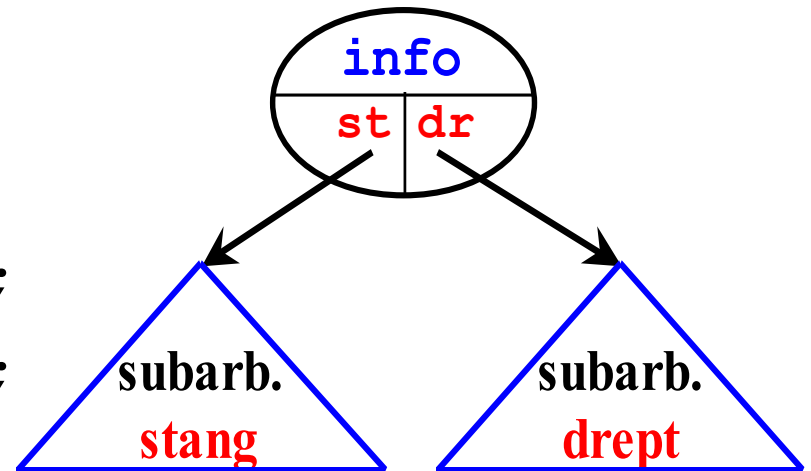
```
    if (!r) return 0;
```

```
    ns = NrNiveluri(r->st);
```

```
    nd = NrNiveluri(r->dr);
```

```
    return 1 + (ns >= nd ? ns : nd);
```

```
}
```



Operatii elementare

```
TArb ConstrNod(TInfo x, TArb s, TArb d);  
    /* -> adresa nod (cu info x,  
                                   fiu stanga s, fiu dreapta d)  
    sau NULL daca nu exista spatiu */  
  
TArb ConstrFr(TInfo x);  
    /* -> adresa frunza cu informatia x,  
    sau NULL daca nu exista spatiu */
```

Creare arbore – constructie frunza

TArb ConstrFr (TInfo x)

**/* -> adresa frunza cu informatia x, sau
NULL daca nu exista spatiu */**

{

TArb aux = (TArb)malloc (sizeof(TNod)) ;

if (!aux) return NULL;

aux->info = x;

aux->st = aux->dr = NULL;

return aux;

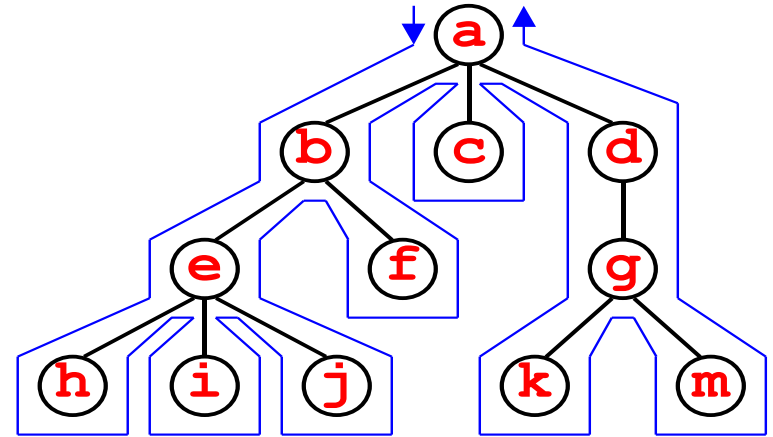
}

Creare arbore – constructie nod

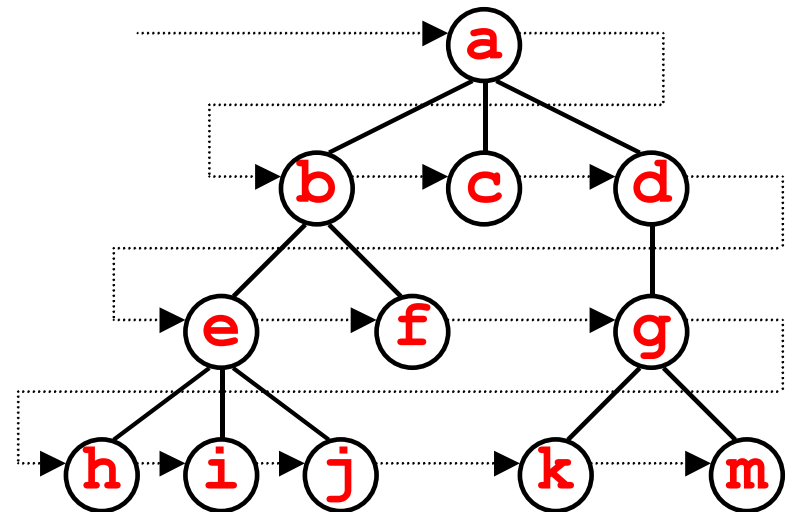
```
TArb ConstrNod(TInfo x, TArb s, TArb d)  
/* -> adresa nod (cu informatia x, fiu stanga s,  
fiu dreapta d) sau NULL in caz de eroare */  
{  
    /* incercare alocare spatiu */  
    TArb aux = (TArb)malloc (sizeof(TNod)) ;  
    if (!aux) return NULL; /* spatiu insuficient */  
  
    /* actualizeaza campurile nodului */  
    aux->info = x; aux->st = s; aux->dr = d;  
    return aux; /* intoarce adresa nodului */  
}
```


Parcurgerea unui arbore

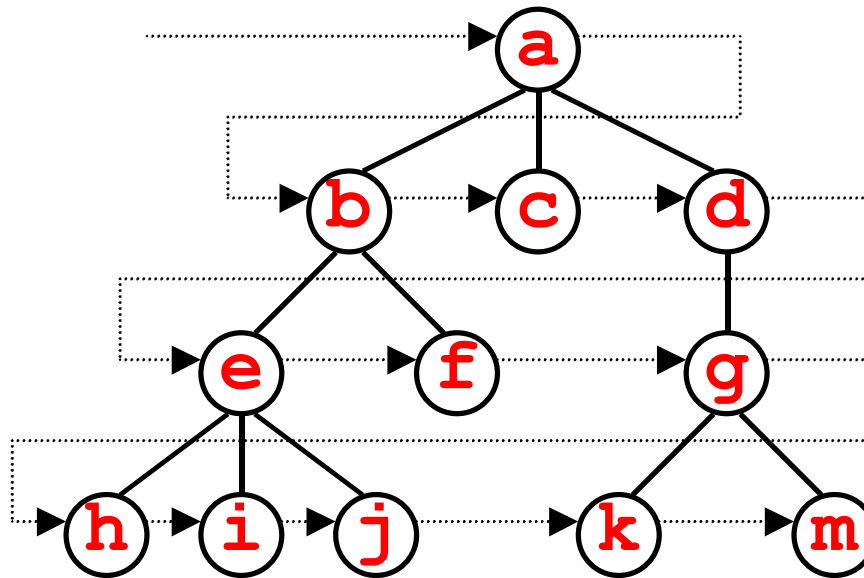
- Adancime



- Latime



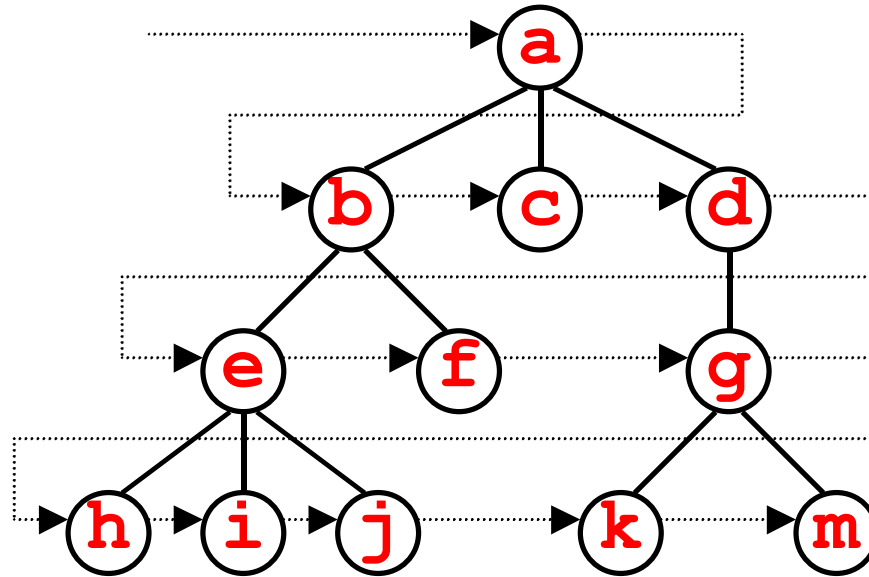
Parcurgerea unui arbore în lăţime



Parcurgere pe nivel:

a, b, c, d, e, f, g, h, i, j, k, m

Parcurgerea unui arbore în lățime



Coadă: a

Parcurgere: a, coada: -

Inserare în coada copii(a), coada: b, c, d

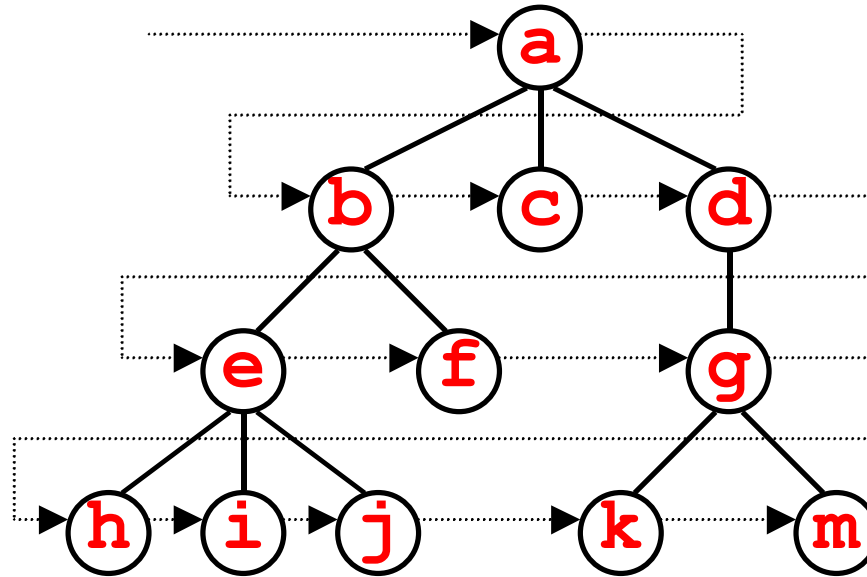
Parcurgere: a, b, coada: c, d

Inserare în coada copii(b), coada: c, d, e, f

Parcurgere: a, b, c, coada: d, e, f

Inserare în coada copii(c), coada: d, e, f

Parcurgerea unui arbore în lăţime



Parcurgere: a, b, c, coada: d, e, f

Inserare in coada copii(c), coada: d, e, f

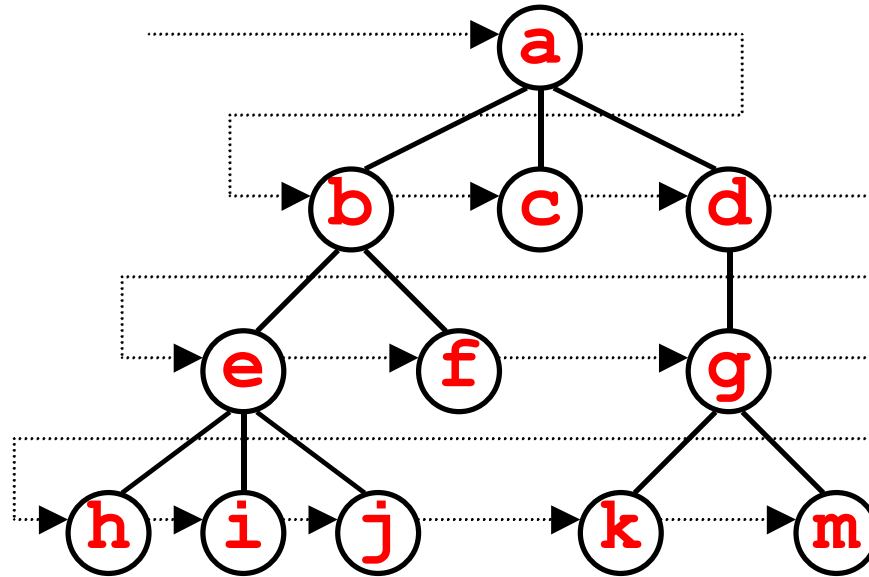
Parcurgere: a, b, c, d, coada: e, f

Inserare in coada copii(d), coada: e, f, g

Parcurgere: a, b, c, d, e coada: f, g

Inserare in coada copii(e), coada: f, g, h, i, j

Parcurgerea unui arbore în lăţime



Parcurgere: a, b, c, d, e coada: f, g

Inserare in coada copii(e), coada: f, g, h, i, j

Parcurgere: a, b, c, d, e, f coada: g, h, i, j

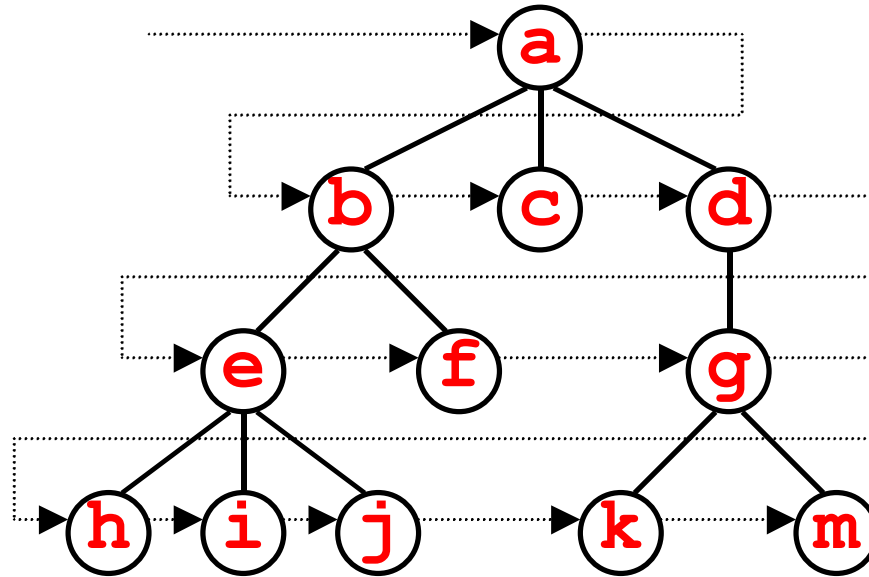
Inserare in coada copii(g), coada: h, i, j, k, m

Parcurgere: a, b, c, d, e, f, g coada: h, i, j, k, m

Inserare in coada copii(h), coada: i, j, k, m

Parcurgere: a, b, c, d, e, f, g, h, coada: i, j, k, m

Parcurgerea unui arbore în lăţime



Parcurgere: a, b, c, d, e, f, g, h, coada: i, j, k, m

Inserare in coada copii(i), coada: j, k, m

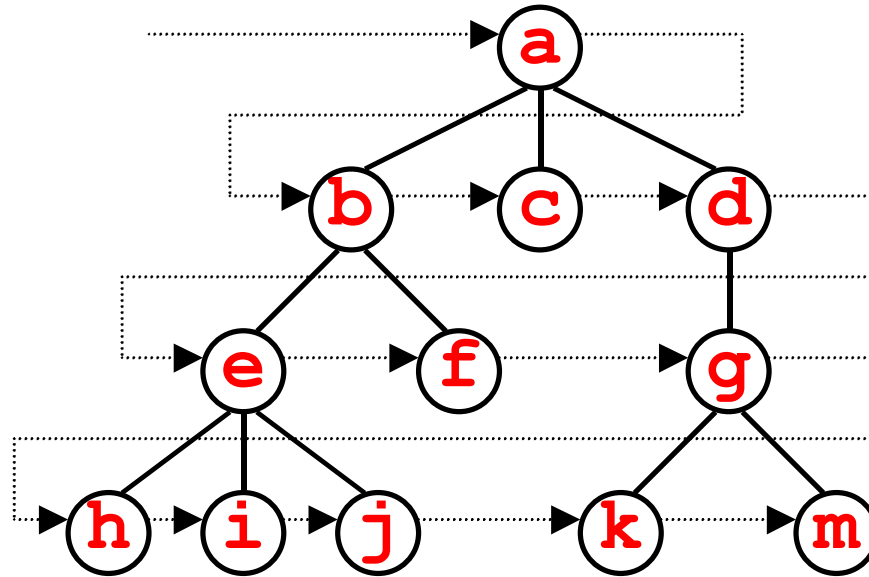
Parcurgere: a, b, c, d, e, f, g, h, i, coada: j, k, m

Inserare in coada copii(j), coada: k, m

Parcurgere: a, b, c, d, e, f, g, h, i, coada: j, k, m

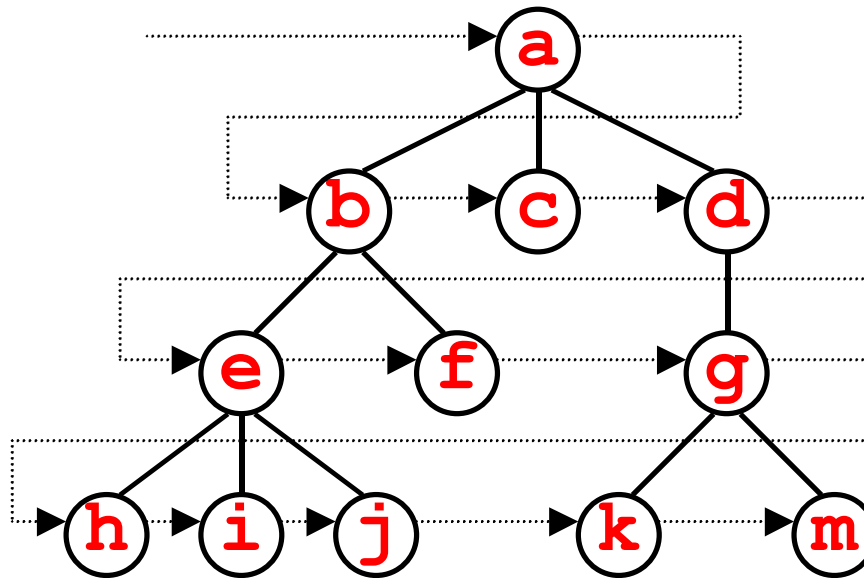
Inserare in coada copii(i), coada: j, k, m

Parcurgerea unui arbore în lăţime



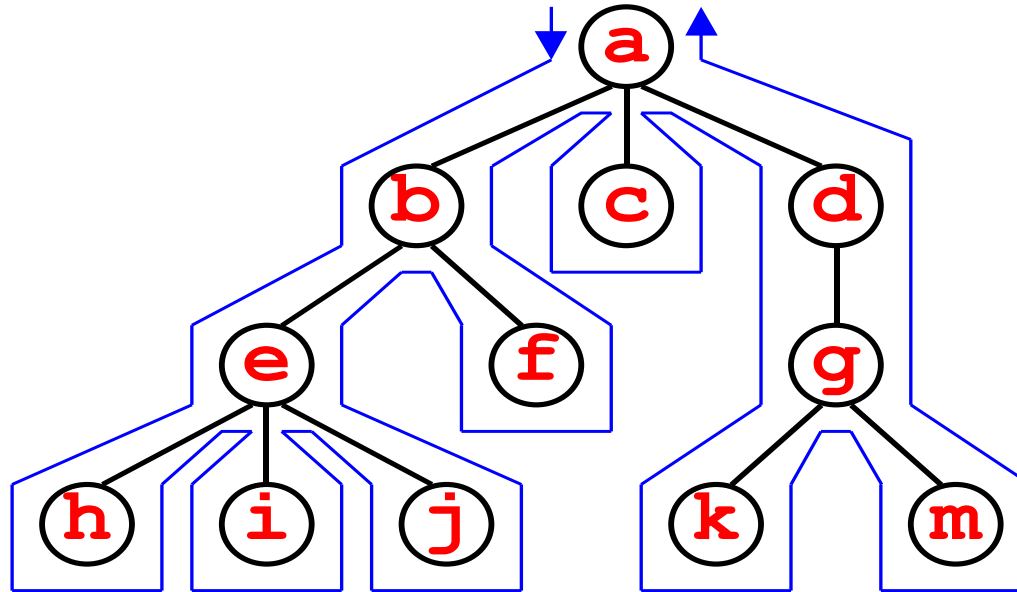
Parcurgere: a, b, c, d, e, f, g, h, i, coada: j, k, m
Inserare in coada copii(i), coada: j, k, m
Parcurgere: a, b, c, d, e, f, g, h, i, j, coada: k, m
Inserare in coada copii(j), coada: k, m
Parcurgere: a, b, c, d, e, f, g, h, i, j, k coada: m
Inserare in coada copii(k), coada: m
Parcurgere: a, b, c, d, e, f, g, h, i, j, k, m coada:-
Inserare in coada copii(m), coada: -

Parcurgerea unui arbore în lăţime



```
inițializează coadă;  
inserează adresa rădăcinii în coadă;  
cât timp coadă nevidă  
{ extrage adresă arbore din coadă;  
  prelucrează informația din rădăcină;  
  pentru fiecare subarbore  
    inserează adresa în coadă;  
}
```


Parcurgerea unui arbore în adâncime

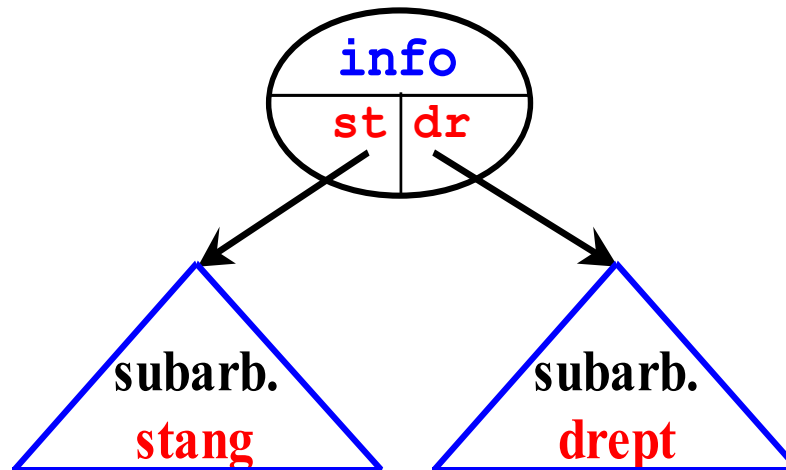


```
Parcurge_in_Adancime (arbore) :  
{ daca arbore vid atunci return;  
  altfel  
  { prelucreaza informatia din radacina;  
    pentru fiecare subarbore  
      Parcurge_in_Adancime (subarbore) ;  
  }  
}
```

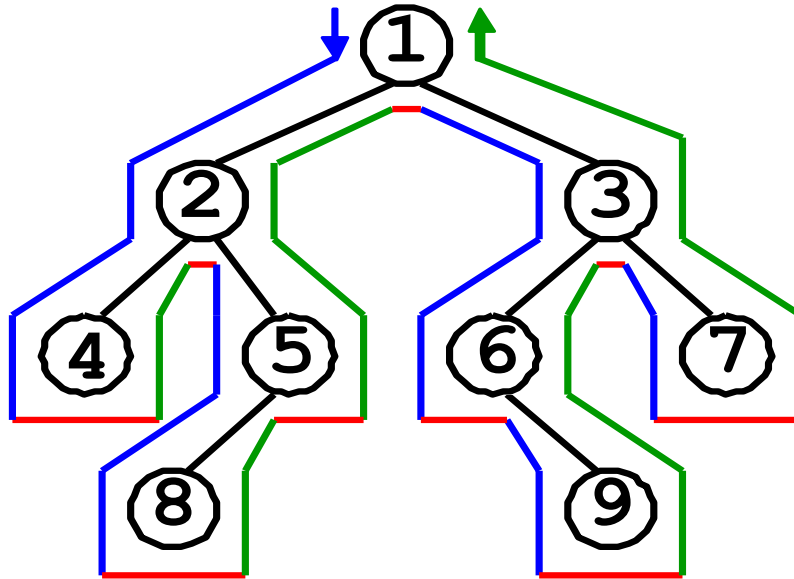
Parcurgerea unui arbore în adâncime

3 tipuri principale de parcurgeri

- **Preordine:** R, S, D
- **Postordine:** S, D, R
- **Inordine:** S, R, D

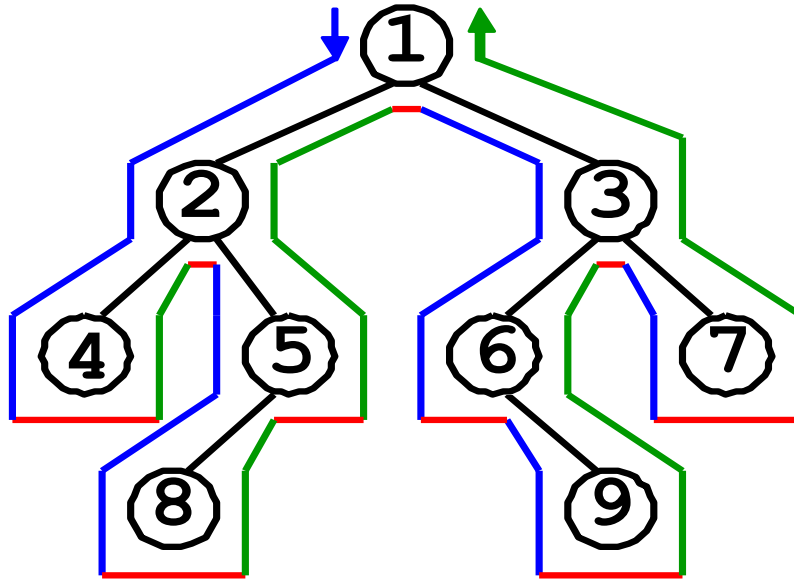


Parcurgeri în adâncime pentru arbore binar



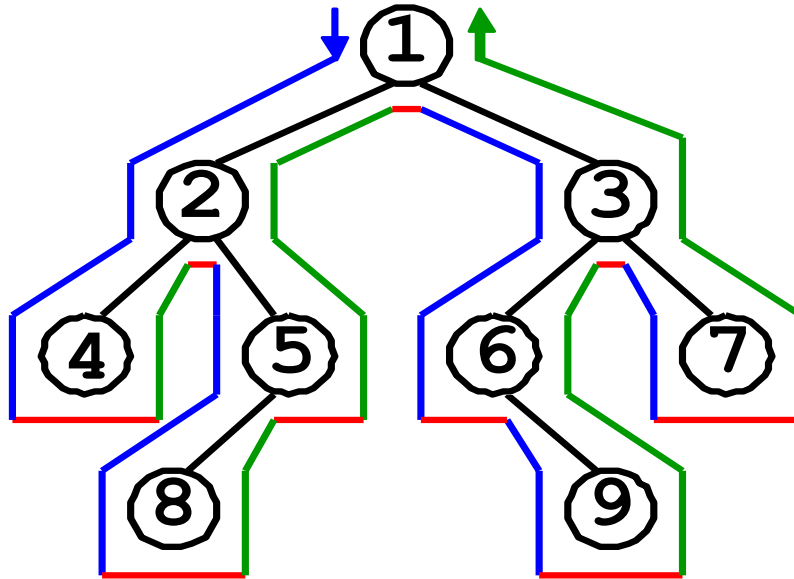
Radacina(Stanga,Dreapta) - in preordine
1, 2, 4, 5, 8, 3, 6, 9, 7

Parcurgeri în adâncime pentru arbore binar



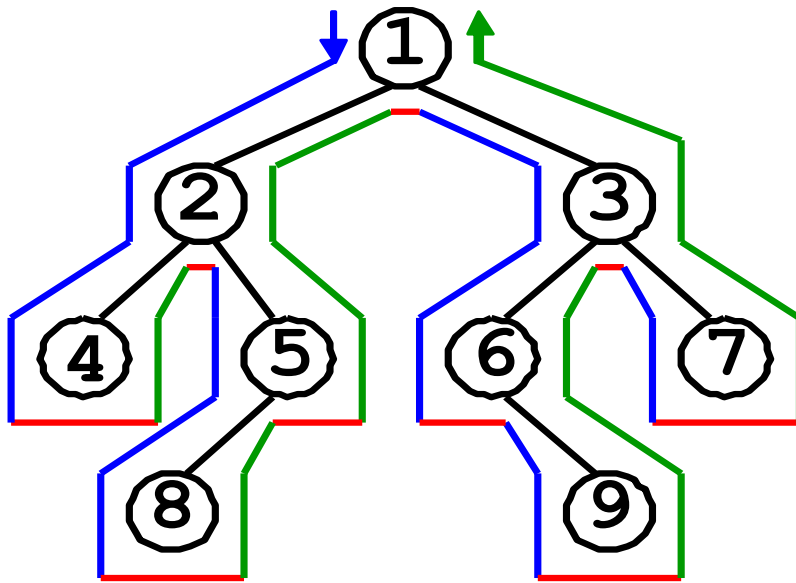
Stanga Radacina Dreapta- in inordine
4, 2, 8, 5, 1, 6, 9, 3, 7

Parcurgeri în adâncime pentru arbore binar



Stanga, Dreapta, Radacina - in postordine
4, 8, 5, 2, 9, 6, 7, 3, 1

Parcurgeri în adâncime pentru arbore binar



Parcursere în preordine
Radacina (Stanga, Dreapta)
1, 2, 4, 5, 8, 3, 6, 9, 7

```
void RSD(Tarb a)
{
    if(!a) return;
    printf("%d ", a->info;
    RSD(a->st);
    RSD(a->dr);
}
```

Parcurgeri în adâncime pentru arbore binar

Numarare noduri din arbore, in parcurgere RSD

```
void RSD(TArb a)
```

```
{  
    if(!a) return;  
    printf("%d ", a->info;  
    RSD(a->st);  
    RSD(a->dr);  
}
```

```
int Numara_RSD(TArb a)
```

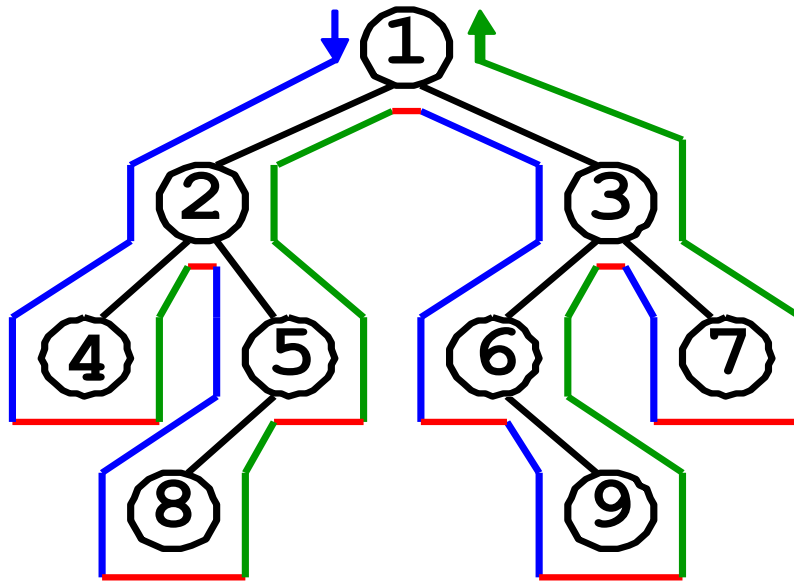
```
{  
    if(!a) return 0;  
    return 1 + RSD(a->st) +  
            RSD(a->dr);  
}
```

Parcurgeri în adâncime pentru arbore binar

Numarare noduri din arbore, in parcurgere SRD

```
int Numara_SRD (TArb a)
{
    int n;
    if(!a) return 0;
    n = Numara_SRD(a->st);
    n++; /* numara radacina */
    n += Numara_SRD(a->dr);
    return n;
}
```


Parcurgeri în adâncime pentru arbore binar



Radacina(Stanga, Dreapta) - in **preordine**

1 (2 (4, 5 (8, -)), 3 (-, 9), 7))

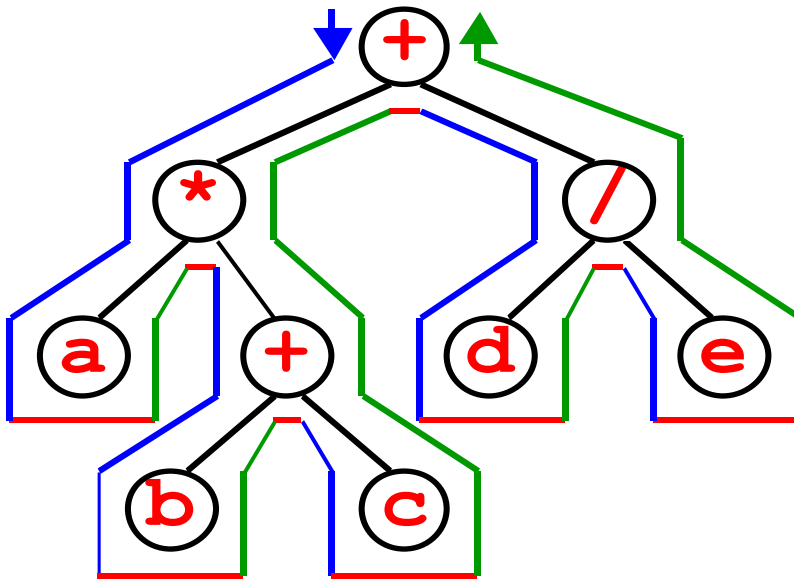
(Stanga **Radacina** Dreapta) - in **inordine**

((4 **2** (8 **5** -)) **1** ((- **6** 9) **3** 7))

(Stanga, Dreapta)**Radacina** - in **postordine**

((4, (8, -) **5**) **2**, ((-, 9) **6**, 7) **3**) **1**

Parcurgeri în adâncime pentru arbore binar



Rădăcină(Stânga, Dreapta) - în **preordine**

+ (* (a, + (b, c)), / (d, e))

(Stânga *Rădăcină* Dreapta) - în **inordine**

((a * (b + c)) + (d / e))

(Stânga, Dreapta) *Rădăcină* - în **postordine**

((a, (b, c) +) *, (d, e) /) +

Parcuregere RSD

```
void RSD(TArb a)
{
    if(!a) {printf("-"); return;}
    if(!a->st && !a->dr)
        {printf(" %d ", a->info); return;}
    printf(" %d ", a->info);
    printf(" (");
    RSD(a->st);
    printf(",");
    RSD(a->dr);
    printf(")");
}
```

Distrugere arbore

```
void distruge(TArb r)
```

```
/* functie auxiliara - distruge toate nodurile */
```

```
{
```

```
    if (!r) return;
```

```
    /* distruge subarborele stang */
```

```
    distruge (r->st);
```

```
    /* distruge subarborele drept */
```

```
    distruge (r->dr);
```

```
    free (r);           /* distruge nodul radacina */
```

```
}
```

Creare arbore

```
void DistrArb(TArb *a)
```

```
/* distruge toate nodurile arborelui de la  
adresa a */
```

```
{
```

```
    if (!(*a)) return;
```

```
    /* distruge toate nodurile */
```

```
    distruge (*a);
```

```
    *a = NULL;
```

```
}
```

Cutare informatie

```
int Cauta(TArb a, int x)
{
    if (!a) return;

    if(a->info == x) return 1;

    int r = Cauta(a->st, x);
    if(r) return 1;

    return Cauta(a->dr, x);
}
```

Cautare informatie + nivelul pe care se afla

```
int Cauta(TArb a, int x)
```

```
{  
    if (!a) return -1;  
    n = aux(a, x, 0);  
    return n;  
}
```

```
int aux(TArb a, int x, int niv)
```

```
{  
    if(!a) return -1;  
    if(a->info == x) return niv;  
    int r = aux(a->st, x, niv+1);  
    if(r >= 0) return r;  
    return aux(a->dr, x, niv+1);  
}
```

Cautare informatie maxima

```
int Max(TArb a) //a - arbore nevid
```

```
{ int m, t;
```

```
  m = a->info;
```

```
  if(a->st) {
```

```
    t = Max(a->st);
```

```
    if(t > m) m = t;
```

```
  }
```

```
  if(a->dr) {
```

```
    t = Max(a->dr);
```

```
    if(t > m) m = t;
```

```
  }
```

```
  return m;
```

```
}
```


Cautare informatie maxima + nivel

```
int Max(TArb a, int* max)
{
    int nivm;
    if(!a) return -1;

    *max = a->info;
    nivm = 0;
    aux(a, 0, max, &nivm);
    return nivm;
}
```

Cautare informatie + nivel

```
void aux(TArb a, int nivCrt, int* max, int* nivM)
{
    if(!a) return;
    if(a->info >= *max) {
        *max = a->info;
        *nivM = nivCrt;
    }
    aux(a->st, nivCrt+1, max, nivM);
    aux(a->dr, nivCrt+1, max, nivM);
}
```

Media informatiilor din arbore

```
float Media(TArb a, int* n)
{
    int suma = 0;
    *n = 0;

    if(!a) return 0;
    aux(a, &suma, n);
    if(n) return (float)suma/(*n);
}
```

Media informatiilor din arbore

```
void aux(TArb a, int* suma, int* n)
{
    if(!a) return;

    (*suma) += a->info;
    (*n)++;

    aux(a->st, suma, n);
    aux(a->dr, suma, n);
}
```

Media informatiilor de pe nivelurile > niv

```
float Media(TArb a, int* n, int niv)
{
    int suma = 0;
    *n = 0;

    if(!a) return 0;
    aux(a, &suma, n, 0, niv);
    if(*n) return (float) suma/ (*n);
}
```

Media informatiilor de pe nivelurile $> n$

```
void aux(TArb a, int* suma, int* n,
        int nivCrt, int niv)
{
    if(!a) return;

    if(nivCrt > niv)
    {
        (*suma) += a->info;
        (*n)++;
    }

    aux(a->st, suma, n, nivCrt+1, niv);
    aux(a->dr, suma, n, nivCrt+1, niv);
}
```

Cautare informatie + nivel

```
int Cauta(TArb a, int x)
```

```
/* intoarce nivelul pe care se afla x sau -1 */  
{ int r;  
  if(!a) return -1;  
  if(a->info == x) return 0;  
  r = Cauta(a->st, x);  
  if(r < 0)  
  {  
    r = Cauta(a->dr, x);  
    if(r < 0) return -1;  
  }  
  return r+1;  
}
```